

Pocket Arcade

Code Review And Capability Inventory - generated July 4, 2026

Scope reviewed: Vite/TypeScript/Phaser app, five game logic engines, Phaser scene renderers, shared input/audio/storage/test bridge systems, unit tests, smoke tests, and validation scripts.

Overall status: the project builds, unit tests pass, lint/import-boundary/format checks pass, and Playwright smoke tests pass after allowing the local Vite server to bind 127.0.0.1:4173. The architecture is clean for an MVP arcade, but several scenes currently under-render the richer logic state, which makes those games less playable than their logic modules suggest.

Validation Results

Check	Result	Notes
npm run build	Pass	TypeScript no-emit succeeded and Vite produced a production build. Vite warned that the JS chunk is 1,220.31 kB minified / 326.46 kB gzip, above the 500 kB warning threshold.
npm run test	Pass	5 Vitest files passed; 22 tests passed.
npm run lint	Pass	ESLint, custom import-boundary script, and Prettier check passed. Import boundary covered 10 logic/test files.
npm run test:e2e	Pass	4 Playwright smoke tests passed and 2 viewport-specific tests were skipped as intended. Initial sandbox run failed with EPERM while binding 127.0.0.1:4173; rerun with local-server permission passed.

Findings

High

Several games simulate real entity positions but render only counts or placeholders.

src/games/star-courier/StarCourierLogic.ts:60-72, 90-116; src/games/star-courier/StarCourierScene.ts:28-33; src/games/lane-rush/LaneRushLogic.ts:55-63, 79-85; src/games/lane-rush/LaneRushScene.ts:27-30; src/games/circuit-stack/CircuitStackScene.ts:26-27

Impact: Star Courier tracks projectile/enemy x-y positions and collision state, Lane Rush tracks traffic lane/y/scored state, and Circuit Stack tracks piece cells. The scenes do not expose or draw those actual positions. Players see aggregate/projected graphics that can diverge from collision truth, so these modes are technically running but not fully readable or fair as games.

Recommendation: Expose render-safe entity arrays or richer snapshots from each logic module, then draw live projectiles, enemies, traffic cars, and the full active falling piece. Add a Playwright or logic-render contract test that checks rendered objects correspond to snapshot state.

Medium

High-score persistence exists as a helper but is not wired into gameplay.

src/core/ScoreManager.ts:3-42; repo search shows no imports of ScoreManager outside its own file

Impact: The README architecture says ScoreManager and Storage handle high-score persistence, and the helper does implement localStorage-backed high scores. Current games maintain private score fields and never instantiate ScoreManager, so users will not actually keep high scores between sessions.

Recommendation: Either integrate ScoreManager into BaseGameScene or each logic engine, or revise the docs until the feature is connected. Add tests for high-score save/read behavior with a localStorage mock.

Medium

Audio unlock listeners can accumulate when switching scenes before the first user gesture.

src/games/BaseGameScene.ts:15, 30-45; src/core/AudioEngine.ts:8-24

Generated July 4, 2026

Impact: Each scene owns an AudioEngine and calls attachUnlockListeners during create. BaseGameScene removes semantic input and restart listeners on shutdown, but AudioEngine has no teardown API for its pre-unlock pointer/key/touch listeners. Repeated game switching before an unlock can leave multiple pending global listeners.

Recommendation: Return a cleanup function from attachUnlockListeners or add an AudioEngine.detachUnlockListeners method and call it in the scene shutdown handler. A singleton shared audio service would also fit the app shell.

Low

Browser smoke tests prove load and switching, but not visual correctness or per-game interaction depth.

tests/smoke.spec.ts:26-76

Impact: The E2E suite confirms the shell loads, Neon Serpent responds, selectors work, and mobile controls are visible. It would not catch the under-rendered Star Courier, Lane Rush, or Circuit Stack states because it only asserts canvas visibility when switching games.

Recommendation: Add one interaction smoke test per game that asserts meaningful state changes through window.__ARCADE__, and consider canvas screenshot or pixel checks for hazards/projectiles/pieces once render snapshots are exposed.

Low

Production bundle is functional but large for a small arcade shell.

npm run build output; README Current Limitations

Impact: The single JS chunk is over Vite/Rolldown warning thresholds because Phaser and all scenes ship together. This is acceptable for an MVP but can slow first load on mobile networks.

Recommendation: Split game scenes with dynamic imports or tune bundling once the MVP behavior stabilizes.

What The Current Code Can Do

The app currently delivers a responsive, zero-external-asset browser arcade shell with five selectable Phaser games. It uses procedural graphics, synthesized WebAudio cues after a user gesture, keyboard and virtual touch controls, deterministic TypeScript logic engines, and test bridge snapshots for browser tests.

Area	Current capability
Application shell	Creates a three-panel desktop layout and compact mobile layout; starts Neon Serpent by default; lets users switch among five games; exposes a restart button and mobile game picker.
Input	Maps arrow/WASD keys, Space/Enter action, Escape/P pause, and touch D-pad buttons into semantic arcade input events.
Rendering	Uses Phaser Canvas with pixel-art settings, responsive resize scaling, and procedural Graphics primitives. No image, font, audio, or sprite-sheet downloads are required.
Audio	Synthesizes select, score, hit, and game-over cues with WebAudio oscillators after unlock.
Testing architecture	Logic engines are deterministic and framework-free. TestBridge exposes window.__ARCADE__.getState() for Playwright assertions. A custom import-boundary script guards logic/test files from Phaser, DOM, localStorage, and audio dependencies.
Persistence	SafeStorage and ScoreManager can persist numeric high scores to localStorage, but the running games do not currently use them.

Game Capability Inventory

Pocket Arcade code review and capability inventory		Generated July 4, 2026
Game	Implemented mechanics	Current presentation status
Neon Serpent	Grid snake on an 18 x 24 board; portal wrapping; food spawning away from occupied cells; combo multiplier; combo decay; speed ramp; obstacle growth; self/obstacle collision; restart.	Most complete slice. Scene draws grid, food, obstacles, full snake body, score, length, and multiplier.
Bounce Circuit	Single-screen platformer physics; left/right movement; jump; spike hazard; key pickup; locked door; win and game-over phases; restart.	Playable MVP. Scene draws ground, spike, key, door, and player.
Star Courier	Vertical shooter logic with fixed projectile/enemy pools, player lane movement, firing, enemy spawn cadence, wave scaling, projectile/enemy collision, and game over when enemies pass the player.	Logic is richer than rendering. Scene draws player correctly but projectiles/enemies are placeholder rows based on active counts, not actual positions.
Lane Rush	Three-lane racer logic; lane clamping; deterministic traffic spawning; speed ramp; collision band; near-miss scoring.	Logic is richer than rendering. Scene draws the player lane but traffic cars are placeholders from trafficCount, not the actual lane/y hazards.
Circuit Stack	Falling-block puzzle logic; 8 x 14 grid; deterministic next pieces; horizontal movement; soft drop; rotation with simple wall kick; row clearing; score table; spawn-blocked game over.	Grid and locked cells render, but the active piece is shown as one circle at the origin instead of its full current shape.

Strengths

- Clear separation between pure game logic and Phaser scene rendering. This makes unit tests fast and readable.
- Good validation discipline for an MVP: build, unit tests, lint, custom architecture guard, formatting, and browser smoke tests all exist and pass.
- The app is honest about zero assets and uses procedural drawing/audio throughout.
- Responsive shell covers desktop and mobile interaction paths, with viewport-specific Playwright coverage.
- Deterministic seeded randomness gives the games repeatable behavior for testing and debugging.

Suggested Next Steps

- Make snapshots/render contracts richer for Star Courier, Lane Rush, and Circuit Stack, then draw the actual live entities.
- Wire ScoreManager into the scoring path or remove the claim until persistence is user-visible.
- Add AudioEngine teardown or move audio to an app-level singleton.
- Expand E2E to exercise every game, not just switching and Neon Serpent movement.
- Consider scene-level code splitting after gameplay fidelity is improved.

Reviewed Files

Primary files reviewed include package.json, README.md, src/main.ts, src/style.css, src/core/*, src/ui/*, all five game Logic/Scene/Test files, tests/smoke.spec.ts, vite.config.ts, playwright.config.ts, eslint.config.js, tsconfig.json, and scripts/import-boundary.mjs.